

Experiment No 5

Name: Parth Vijay Gulhane

Roll No: 23CO043

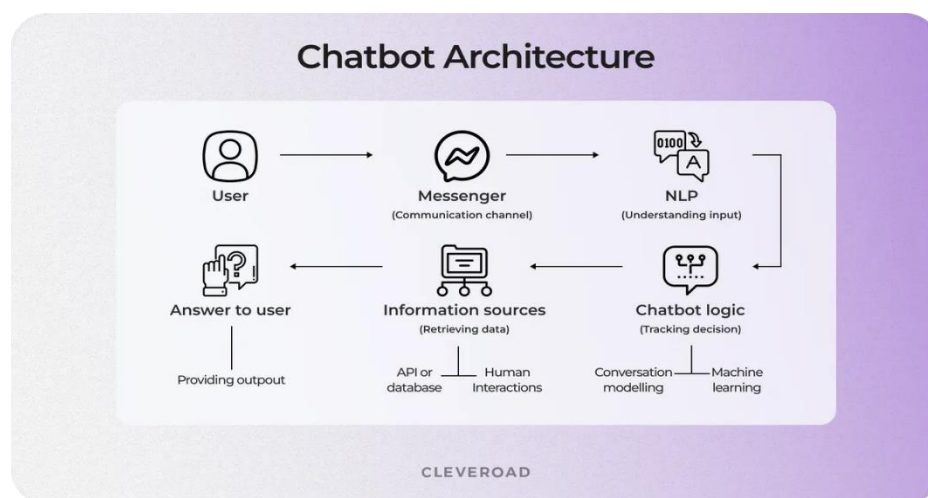
Class: TE Comp A

Aim

To develop an elementary chatbot for a customer interaction application using Natural Language Processing (NLP) techniques in Python.

Objectives

- To understand the fundamentals of Artificial Intelligence (AI) and Natural Language Processing (NLP).
- To implement text preprocessing techniques such as tokenization and lemmatization.
- To design a chatbot capable of simulating human conversation.
- To handle customer-related queries like product details, pricing, and services.
- To apply rule-based systems and pattern matching techniques.
- To explore real-world applications of AI in business automation.



Theory

Artificial Intelligence is a branch of computer science that aims to create systems capable of performing tasks that typically require human intelligence. A chatbot is one such application that enables communication between humans and machines.

Natural Language Processing (NLP)

NLP is a subfield of AI that deals with interaction between computers and human language.

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, and respond to human language.

In this experiment, a rule-based chatbot is developed using NLP concepts from the SPPU syllabus such as tokenization, lemmatization, pattern matching, and knowledge representation.

NLP Techniques Used Technique

Description

Splitting Text Into Words
Tokenization
Converting words to base form
Lemmatization
Standardizing
Lowercasing text input

Working of Chatbot

The chatbot follows a structured pipeline:

1. **User Input** → User enters a query
2. **Preprocessing** → Text is tokenized and lemmatized
3. **Pattern Matching** → Regex patterns are matched
4. **Response Selection** → Appropriate response is selected
5. **Output** → Response displayed to user

Knowledge Representation

The chatbot stores predefined responses in a **knowledge base**.

Example Knowledge Base Table

User Query Pattern Bot Response

Hello / Hi	Welcome message
Price / Cost	Pricing details
Order / Buy	Order assistance
Contact	Contact information

Rule-Based System

The chatbot is based on **if-then rules**, which is a fundamental AI approach.

Rule Type Example

If input contains "price" Show pricing info

If input contains "order" Assist with order

If no match Default fallback response

Pattern Matching using Regular Expressions

Regular expressions (regex) are used to identify keywords.

Example:

```
r'\b(price|cost|fee)\b'
```

This matches words like:

- price
- cost
- fee

System Architecture

User Input → NLP Processing → Pattern Matching → Knowledge Base → Response Output

Features of the Chatbot

Feature	Description
User Interaction	Conversational interface
Name Recognition	Personalized responses
Business Queries	Handles customer support
Fast Response	Instant replies
Simple Design	Easy to implement

Advantages

- Reduces human effort
- Available 24/7
- Fast response time
- Easy to implement
- Improves customer experience

Applications

Domain	Use Case
E-commerce	Customer support
Banking	Query handling
Healthcare	Appointment booking
Education	Student assistance

Algorithm (Simple Steps)

1. Start
2. Accept user input
3. Preprocess text (tokenize, lemmatize)
4. Match input with predefined patterns
5. If match found → return response
6. Else → return default message
7. Repeat until user exits
8. Stop

Conclusion

The developed chatbot demonstrates the implementation of Natural Language Processing concepts such as tokenization, lemmatization, and pattern matching. It successfully handles basic customer interactions and provides automated responses. This experiment shows how NLP can be used in real-world business applications to improve efficiency and user experience.

Code

```
#include <iostream>
#include <string>
#include <map>
#include <regex>
#include <algorithm>
using namespace std;

// Convert string to lowercase
string to_lower(string text) {
    transform(text.begin(), text.end(), text.begin(), ::tolower);
    return text;
}

class FinalBot {
private:
    string user_name;
    map<string, string> kb = {
        {R"(b(hi|hello|hey)\b)", "Hello! Welcome to Customer Support. How can I assist you?"},
        {R"(b(order|status|track)\b)", "Provide your order ID or check 'Track Order' page."},
        {R"(b(return|refund|exchange)\b)", "We have a 30-day return policy."},
        {R"(b(shipping|delivery)\b)", "Shipping takes 3-5 days."},
        {R"(b(payment|billing)\b)", "We accept cards, PayPal, Apple Pay."},
        {R"(b(agent|human)\b)", "Connecting you to a live agent..."},
        {R"(b(hours|time)\b)", "Support: Mon–Fri, 9AM–6PM."},
        {R"(b(password|account)\b)", "Use 'Forgot Password' option."},
        {R"(b(thanks|thank)\b)", "You're welcome!"},
        {R"(b(who are you)\b)", "I am your support assistant."}
    };
public:
    string get_response(string user_input) {
        string processed = to_lower(user_input);
        // Name detection
        if (user_name.empty()) {
            if (processed.find("my name is") != string::npos ||
                processed.find("i am") != string::npos) {
                int pos = user_input.find_last_of(" ");
                user_name = user_input.substr(pos + 1);
                return "Nice to meet you, " + user_name + "!";
            }
            else if (count(user_input.begin(), user_input.end(), ' ') == 0) {
                user_name = user_input;
                return "Nice to meet you, " + user_name + "!";
            }
        }
    }
};
```

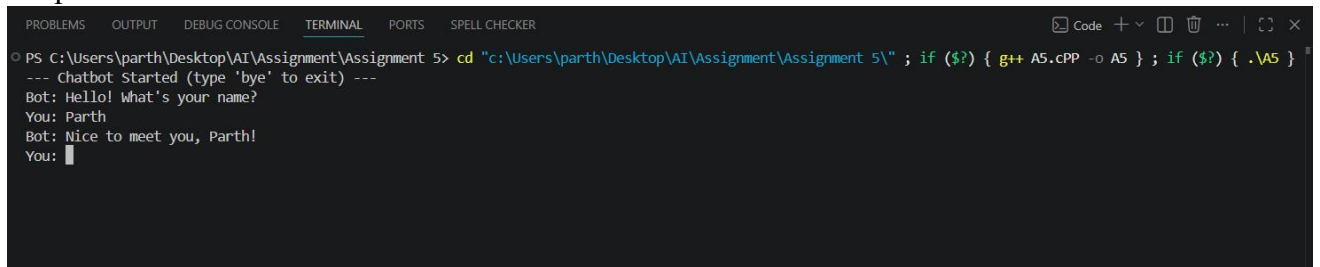
```

    }
}
// Pattern matching
for (auto &entry : kb) {
    regex pattern(entry.first);
    if (regex_search(processed, pattern)) {
        if (!user_name.empty() && entry.second.find("Hello") != string::npos) {
            return "Hello, " + user_name + "! How can I help?";
        }
        return entry.second;
    }
}
return "Sorry " + user_name + ", I didn't understand.";
}
};

int main() {
    FinalBot bot;
    cout << "--- Chatbot Started (type 'bye' to exit) ---" << endl;
    cout << "Bot: Hello! What's your name?" << endl;
    while (true) {
        string user;
        cout << "You: ";
        getline(cin, user);
        string lower = to_lower(user);
        if (lower == "bye" || lower == "exit") {
            cout << "Bot: Goodbye!" << endl;
            break;
        }
        cout << "Bot: " << bot.get_response(user) << endl;
    }
    return 0;
}

```

Output:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 5> cd "c:\Users\parth\Desktop\AI\Assignment\Assignment 5\" ; if ($?) { g++ A5.cpp -o A5 } ; if ($?) { .\A5 }
--- Chatbot Started (type 'bye' to exit) ---
Bot: Hello! What's your name?
You: Parth
Bot: Nice to meet you, Parth!
You: 

```